Lesson 3 — NFV, SDN, Docker and High Availability on AWS

INTE 22253 | Source: Lecture 03 (67 slides, almost all images: NFV, SDN, industry examples, containers and Docker, AWS global infrastructure, high availability, auto scaling, business continuity and disaster recovery) | Builds on Lesson 2

WHAT YOU WILL BE ABLE TO DO AFTER THIS LESSON

- Explain **Network Functions Virtualization (NFV)**, its advantages, and how it differs from a virtualized network and from **SDN**.
- Explain why **containers** exist, the **Docker** architecture (client, daemon, images, containers, registries), Dockerfile layers, the container lifecycle and volumes.
- Describe AWS global infrastructure (**regions, availability zones**) and draw a **highly available 3-tier architecture** with a load balancer, auto scaling groups and a replicated database.
- Explain capacity planning, horizontal vs vertical scaling, launch templates and the three **auto scaling policies**.
- Define **RTO and RPO** and compare the four disaster-recovery strategies: backup & restore, pilot light, warm standby, multi-site active/active.

1. Network Functions Virtualization (slides 1–8)

A **network function** is a job a network device does to traffic: a **firewall** (blocks unwanted traffic), a **router** (forwards packets between networks), a **load balancer**, **NAT** (translates private to public addresses), **encryption/VPN**, intrusion detection. Traditionally each function was a separate, expensive **hardware appliance** (a dedicated box).

DEFINITION (SLIDE 2)

NFV is a way to **reduce cost and accelerate service deployment** for network operators by **decoupling functions like a firewall or encryption from dedicated hardware** and moving them to **virtual servers**.

- If a customer wants a new network function, the provider simply **spins up a new virtual machine** to perform it — no new box to buy and install.
- It reduces dependency on dedicated hardware appliances and improves **scalability and customization** across the network.
- **Different from a virtualized network:** NFV offloads **network functions only**, not the entire network.

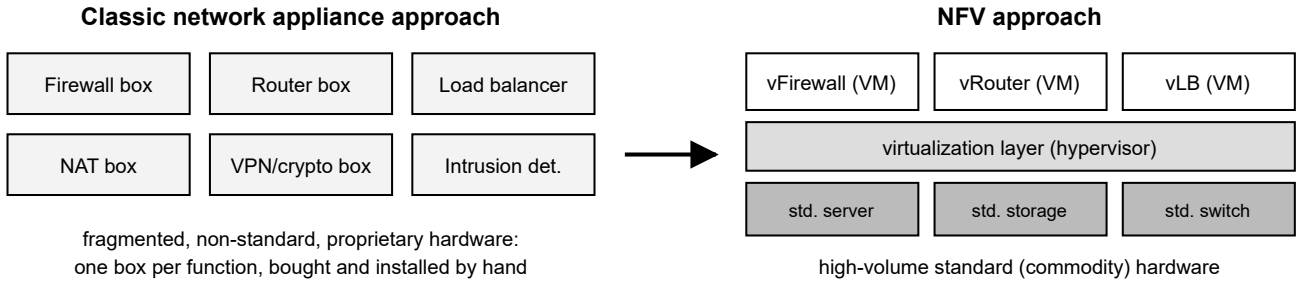


Figure 1 — Slide 3: classic appliances vs NFV.

Advantages of NFV (slide 5)

NFV reduces the need for dedicated hardware to deploy and manage networks, and the functions can be **managed from anywhere** within the operator's network.

- Reduced space needed for network hardware
- Easier network upgrades (update software, not boxes)

- Reduced network power consumption
- Reduced network maintenance costs
- Longer life cycles for network hardware
- Reduced maintenance and hardware costs

NFV in the world (slides 6–8): operators manage and expand capabilities **on demand** using software where physical boxes once stood. It becomes easier to **load-balance, scale up and down, and move functions** across distributed hardware, and continual software updates keep the network current without interrupting customers. Slide 6 shows **Ericsson's NFV stack**: **NFV MANO** (Management and Orchestration: an orchestrator that manages the virtual network functions' life cycle), **VNFs** (Virtual Network Functions, e.g. virtual packet core), and **NFVI** (NFV Infrastructure: data-centre hardware, networking, storage, with SDN and a VIM, the Virtualized Infrastructure Manager). Slide 8 shows **Oracle SDN** virtual network services connecting web, app and DB tiers, each in its own subnet.

2. Software-Defined Networking (slides 9–11, 22)

The slides show SDN industry diagrams and a "Why SDN?" list. Here is the idea behind them.

Every network device has two jobs: the **control plane** decides *where* traffic should go (builds routing tables, applies policies); the **data plane** (forwarding plane) actually *moves* the packets. In traditional networking both live inside every switch and router, and each box is configured one by one. **SDN separates them**: a central software **SDN controller** holds the control plane for the whole network and programs simple forwarding devices.

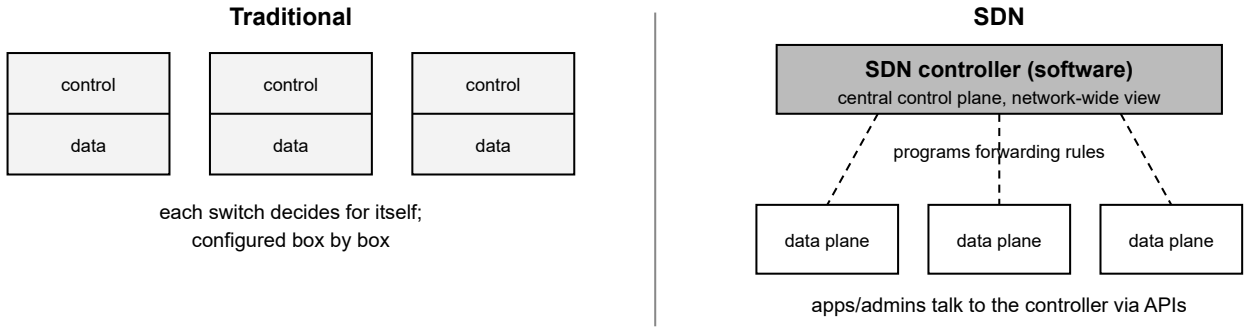


Figure 2 — SDN moves the control plane out of each device into a central controller.

Aspect	NFV	SDN
What is virtualized / changed	Network **functions** (firewall, router, LB) become software on VMs	Network **control** is separated from forwarding and centralized
Main goal	Replace proprietary appliances with commodity servers	Programmable, centrally managed networks
Relationship	Complementary: SDN often connects and steers traffic between NFV functions (slide 6 shows SDN inside the NFV stack). NaaS (Lesson 2) is built on SDN.	

Why SDN? (slide 22) — seven reasons:

1. Manage the costs of WAN links (wide-area links between sites).
2. Improve control of hybrid WAN and multi-cloud environments.
3. Assure service and prioritise business-critical traffic.
4. Introduce new sites and capabilities (quickly, from software).
5. Prevent attacks and mitigate security risks.
6. Manage different network domains and services across the whole enterprise.
7. Future-proof the enterprise's advancing requirements while reducing complexity.

Slides 10–11 show an SDN deployment where each host runs web, app and DB tier VMs connected through software bridges and bonds, plus tenant routers — the network topology is defined in software, not cables. Slide 18 shows an **SD-WAN**: headquarters, a sales office and a home office connected over the Internet to the data centre through managed gateways, controlled from one cloud console.

3. Industry examples (slides 12–21)

Example	What the slides show
Citrix DaaS	Desktop as a Service: Citrix Virtual Apps and Desktops run in the data centre/cloud and are delivered to users. Reduces the lengthy hardware procurement cycle; provides advanced monitoring and cloud-native management to scale fast for a flexible workforce.
Akasa Cloud	A local cloud stack: applications → virtual machines → hypervisor → computing and memory → storage, managed by an SLT management portal. Customers choose firewall rules, networks/IPs, OS types, computing power/RAM and storage size for each VPS (virtual private server).
Dialog Cloud	Desktop as a Service (DaaS: virtual desktops hosted in the cloud), Cloud Backup as a Service (BaaS), Co-location (rent space in Dialog's data centre for your own servers), Disaster Recovery as a Service (DRaaS), Dialog Enterprise Cloud.
Wired media	Structured cabling systems (organised patch panels and racks) in a data centre (slides 20–21).

4. Containers and Docker (slides 23–34)

4.1 Why containers? (slide 26)

Point	Explanation
Challenges of traditional deployment	**Dependency conflicts** (app A needs library v1, app B needs v2 on the same server) and **inconsistent environments** cause the "**works on my machine**" problem: code runs on the developer's laptop but fails on the server.
Container packaging benefits	A container bundles the application with **all its dependencies**, so it behaves the same in development, testing and production.
Support for microservices	Many small independent services can be deployed and scaled efficiently on shared hosts.
Improved software delivery	Fewer environment-related errors; faster development and deployment cycles.

4.2 VMs vs containers (slide 28)

VMs run full guest operating systems on a hypervisor: strong isolation but resource overhead. **Containers** share the host OS kernel using **namespaces** and **cgroups**: lightweight and fast to start. Containers give **higher density** (more apps per server); VMs suit **strict isolation and diverse OS needs**.

- **Namespaces** (Linux kernel feature): give each container its own *view* of the system — its own process list, network interfaces, file system mount points, hostname. A container cannot see other containers' processes.
- **cgroups** (control groups): *limit* how much CPU, memory and I/O a container may use, so one container cannot starve the others.

4.3 Docker architecture and core components (slide 30)

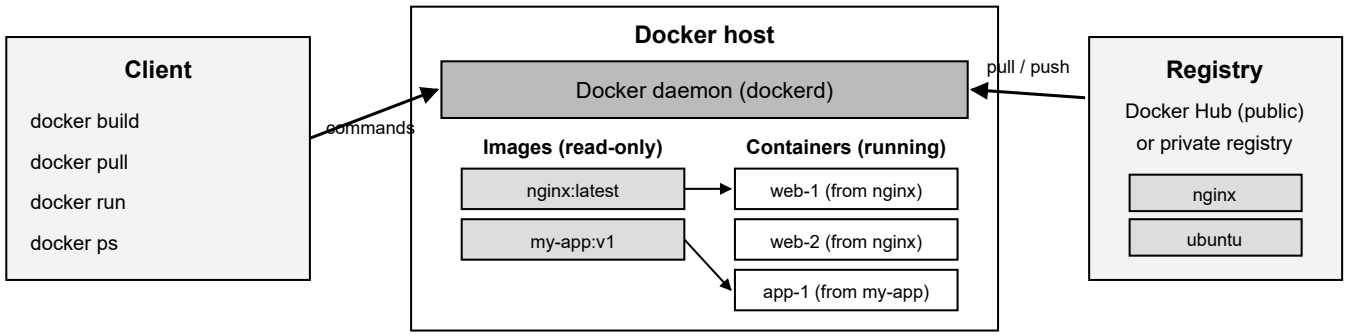


Figure 3 — Docker client-server architecture. An image is a template; containers are running instances of it.

Component	Role (slide 30)
Client-server architecture	The Docker **client** (the `docker` command) sends commands to the Docker **daemon**, which manages containers.
Docker daemon	Builds images, runs containers, manages networks and volumes, and communicates with registries.
Images and containers	An **image** is a **read-only template** (app + dependencies + settings). A **container** is a **running instance** created from an image. Analogy: image = class, container = object; or image = recipe, container = the cooked dish. Many containers can come from one image.
Image registries	Centralized storage and sharing of images, public (Docker Hub) or private.

4.4 Dockerfiles, layers, lifecycle, volumes (slide 32)

Images are built from a **Dockerfile**: a text file with step-by-step instructions that specify the image's configuration and contents. **Each instruction creates a new image layer**; layers are **cached**, so a rebuild only redoes the layers that changed (efficient incremental builds).

Dockerfile (example)	What each line does
`FROM python:3.12-slim`	Start from a base image (layer 1): a minimal Linux with Python
`WORKDIR /app`	Set the working folder inside the image
`COPY requirements.txt .`	Copy the dependency list (new layer)
`RUN pip install -r requirements.txt`	Install libraries (new layer, cached until requirements.txt changes)
`COPY . .`	Copy the source code (new layer; changes often, so it is last)
`CMD ["python", "app.py"]`	Command run when a container starts

```
docker build -t my-app:v1 .          # build image from the Dockerfile in this folder
docker run -d -p 8080:80 my-app:v1  # create + start a container, map host port 8080 to container port 80
docker ps                           # list running containers
docker stop <id> ; docker rm <id>  # stop, then remove the container
docker push myrepo/my-app:v1        # upload the image to a registry
```

Container lifecycle: created → started (running) → stopped → restarted → removed. Containers are designed to be **ephemeral** (short-lived and disposable): you replace them rather than repair them.

Persistent data: anything written inside a container is lost when it is removed. Data that must survive is stored **outside** the container in a **volume** (e.g. `docker run -v dbdata:/var/lib/mysql mysql`).

4.5 Use cases, benefits and limitations (slide 34)

- **Application packaging**: apps bundled with dependencies give consistent dev/test environments.
- **Production deployment**: faster, reliable deployments with rolling updates and horizontal scaling of services.

- **Security limitation**: containers share the host OS kernel, so a kernel exploit could affect all containers; careful security practice is needed compared with VMs.
- **Efficiency and portability**: faster release cycles across environments.

5. AWS global infrastructure (slides 35–37)

Lecture overview: AWS global infrastructure, disaster recovery strategies, recovery point objectives, maintaining business continuity, Route 53 routing policies, AWS migration services.

- **Regions**: located all around the world; servers and services are based in regions; **choose a region based on user locations** (and law/compliance).
- **Availability Zones (AZs)**: a cluster of data centres with **independent power and cooling** and **multiple redundant network connections**. **All regions have at least 2 AZs.** If you deploy in one data centre only, you get the "single data centre experience": one failure takes you down.

Eliminating single points of failure (slide 37): a basic app is a chain **front-end web server → application server → database server**. If any one box fails, the whole app fails, so each one is a **single point of failure (SPOF)**. The fix is redundancy at every tier, in more than one AZ.

6. Highly available solution architecture (slides 38–41, 48, 51)

The lecture builds the design in four steps:

1. Put web front-end, app server and an **RDS primary** database in a **private subnet** in AZ 1, and copies plus an **RDS secondary** in AZ 2; the database **replicates** primary → secondary.
2. Add **public subnets** with a **NAT gateway** in each AZ (lets private servers reach the Internet for updates without being reachable from it).
3. Add an **Application Load Balancer (ALB)** spanning both AZs to receive user traffic.
4. Put the web and app tiers into **Auto Scaling groups** that span both AZs.

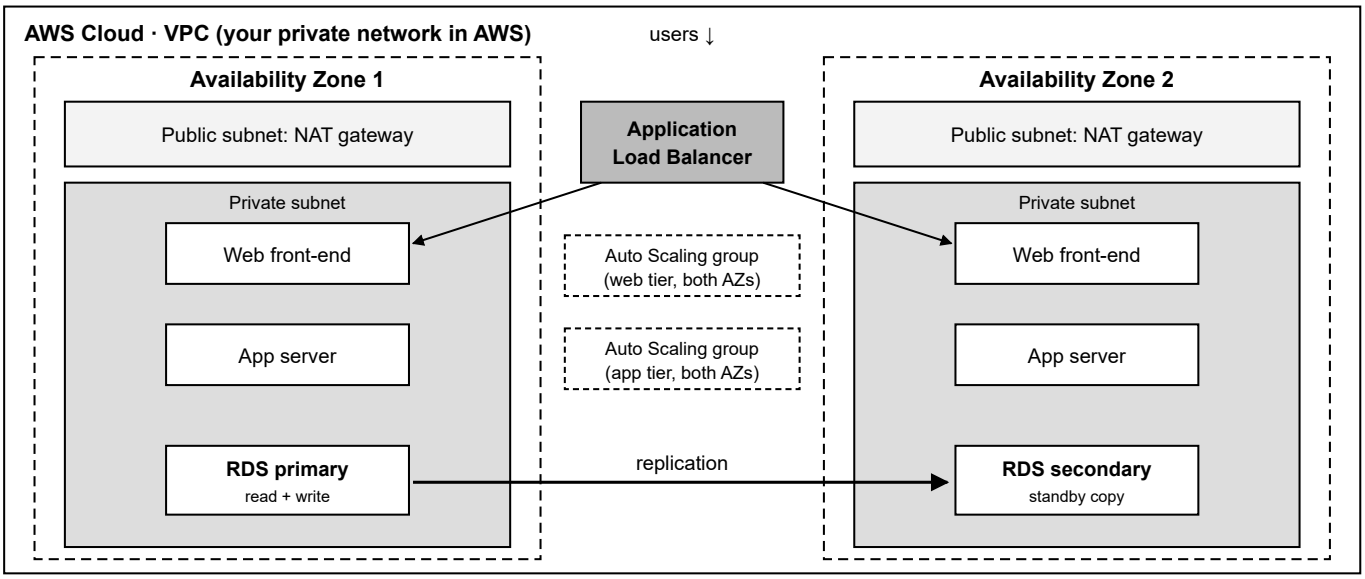


Figure 4 — Highly available 3-tier architecture across two AZs (slides 41/48). If AZ 1 fails, the ALB sends all traffic to AZ 2, the Auto Scaling group adds instances there, and RDS fails over to the secondary.

Building block	Meaning	HA role
VPC	Virtual Private Cloud: your isolated private network inside AWS	Holds all resources
Public / private subnet	Public subnet has a route to the Internet; private subnet does not	Servers and DB hidden from the Internet
NAT gateway	Lets private instances make outbound connections only	One per AZ so losing an AZ does not cut the other off
Application Load Balancer	Spreads requests over healthy instances in several AZs	Removes the web server SPOF
Auto Scaling group	Logical collection of instances kept at a target size	Replaces failed instances, adds capacity under load
RDS primary + secondary	Managed database with a synchronously replicated standby in another AZ (Multi-AZ)	Removes the database SPOF

3-tier architecture (slide 51): presentation tier (front-end web server) → logic tier (application server) → data tier (database server). Each tier can be scaled and protected separately.

7. Capacity planning and scaling (slides 42–50)

7.1 Capacity planning (slide 42)

Capacity planning = deciding how much compute to provide for the expected demand. The slide's four charts (bars = demand, shading = available capacity):

Case	What happens
Expectation	Demand is mostly low with occasional tall spikes
Too much capacity	Capacity sized for the peak: the spike is served, but most of the time you pay for idle servers (waste)
Not enough capacity	Capacity sized for normal load: the spike exceeds it, users get slow responses or errors (lost business)
Auto scaling	Capacity follows demand: grows for the spike, shrinks afterwards — no waste, no shortage

7.2 Horizontal and vertical scaling (slides 43–44)

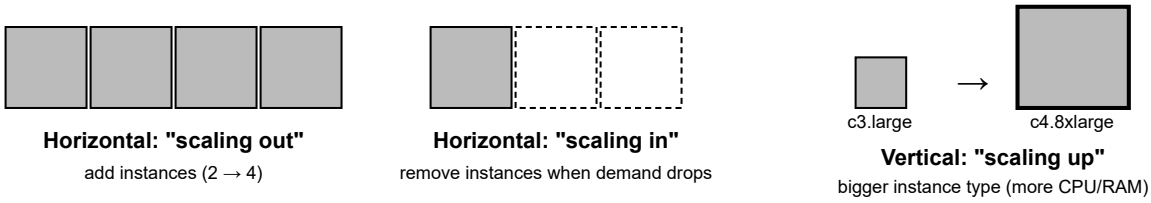


Figure 5 — Scaling out / in (horizontal) and up (vertical) (slides 43–44).

Vertical scaling usually needs a **restart** onto a bigger instance and has an upper limit; horizontal scaling adds availability because there are more copies. Cloud designs prefer horizontal scaling.

7.3 Launch templates (slides 45–46)

A **launch template** stores everything needed to start an instance: machine image (AMI), instance type, network, security groups, and **user data** (a startup script, e.g. install and start a web server). Auto Scaling uses it to create identical instances. Slide points: **recommended by AWS over launch configurations**; **can be changed and have multiple versions**; **allow more advanced configuration options**. Slide 46 shows user data that starts `httpd` and writes the instance's availability zone into `index.html`.

7.4 Auto Scaling (slides 47–50)

Auto Scaling: **right-sizes capacity needs**; an Auto Scaling group is a **logical collection of instances**; instances are **spread across Availability Zones**; it **performs health checks** on instances and replaces unhealthy ones. You

set **minimum, maximum and desired capacity** — manually, or automatically from metric values.

Policy	How it works	Example
Target tracking	Auto Scaling keeps a metric at a **target value** (like a thermostat)	"Keep average CPU at 50%": at 80% it adds instances, at 20% it removes some
Simple scaling	Acts on a **CloudWatch alarm** (AWS monitoring): one fixed action, then waits a cooldown	"If CPU > 70% for 5 min, add 1 instance"
Step scaling	Acts on **banded value ranges**: bigger breach ⇒ bigger step	CPU 70–85% add 1; 85–95% add 2; >95% add 4

Worked example (step scaling): min 2, max 10, currently 3 instances. CPU rises to 90% ⇒ band 85–95% ⇒ add 2 ⇒ 5 instances. CPU then 97% ⇒ add 4 ⇒ 9. Next band would give 13, but max is 10 ⇒ capped at **10**. When CPU falls, a scale-in rule removes instances, never going below 2.

7.5 RDS vs running a database on EC2 (slides 52–53)

Amazon RDS (managed)	Database on EC2 (self-managed)
Fully managed; AWS handles patching and maintenance; automated backups for **point-in-time recovery**; user-initiated snapshots; **Multi-AZ deployments**; leverages EC2 instances and EBS volumes underneath	You get shell access to the server file system and other server-side dependencies; you must be willing to spend DBA (database administrator) labour in exchange for lower usage costs; additional work to support maintenance and failover scenarios

8. Business continuity and disaster recovery (slides 54–61, 65)

Business continuity = keeping the business running during and after a disruption. **Disaster recovery (DR)** = the IT plan to restore systems and data after a disaster (region outage, flood, ransomware). Two numbers drive the choice:

RPO AND RTO (LEARN BOTH)

RPO — Recovery Point Objective: the maximum amount of **data loss** you can accept, measured in time. "How far back in time is the last good copy?" Backups every 24 h ⇒ RPO up to 24 h.

RTO — Recovery Time Objective: the maximum **downtime** you can accept. "How long until we are running again?"

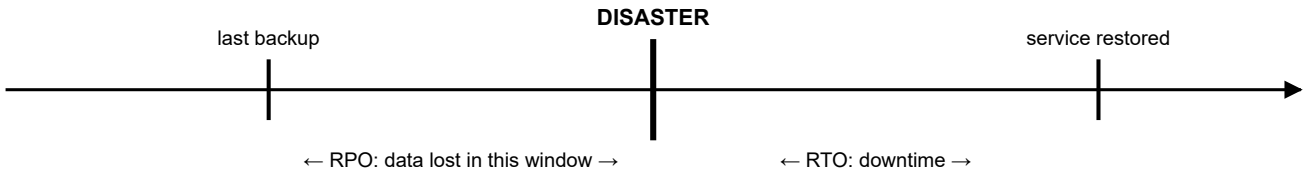


Figure 6 — RPO looks back from the disaster (data loss); RTO looks forward (downtime).

The four DR strategies (slides 55–61)

Strategy	What is running in the DR region before the disaster	What you do after	RPO / RTO	Cost
Backup & restore	Nothing — only regularly scheduled backups, copied across regions on a schedule. No standby infrastructure.	Manually spin up new infrastructure and restore the backups	Hours	$
Pilot light	Only the core: the **data is live** (DB read replica receiving replication); app/web servers **idle or off**	Provision/switch on the app tier, scale it, promote the replica, point DNS (Route 53) to it	10s of minutes	$$
Warm standby	A **scaled-down but fully working** copy always running (small ALB, web, app, read replica)	Scale it up to full production size and fail over	Minutes	$$$
Multi-site (active/active)	Full production in **both** regions, both serving users; Route 53 splits traffic	Nothing to start — traffic just goes to the surviving site	Real time (near-zero data loss, zero downtime)	$$$$

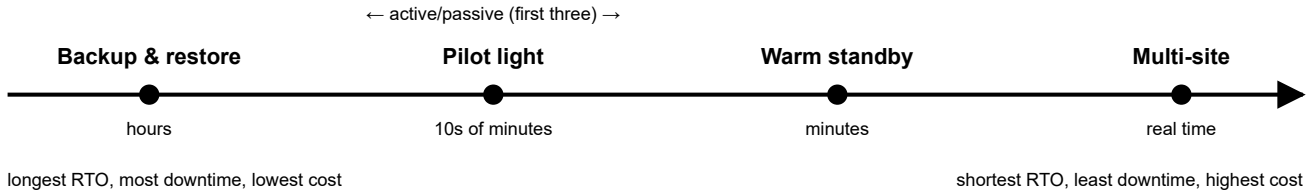


Figure 7 — Slides 55 and 61: DR options from cheapest/slowest to most expensive/fastest. The first three are active/passive; multi-site is active/active.

Cost trade-off (slide 65): two curves against the length of service interruption. The **cost of business impact** rises the longer you are down; the **recovery cost** (cost and complexity of the DR solution) falls as you accept longer recovery. The **RTO** you choose is where an acceptable recovery cost meets an acceptable business impact. Multi-site is at the expensive end, backup & restore at the cheap end.

Route 53 is AWS's DNS service; in the DR diagrams it points the domain name (e.g. carvedrockfitness.com) at the active region and switches it during failover (failover routing). The lecture overview also lists AWS migration services, without detail.

Worked example: choose a DR strategy

Requirement	Choice	Reason
Internal HR wiki; can be down a day; tiny budget	Backup & restore	RTO/RPO of hours is acceptable; cheapest
Online shop; down at most ~30 min; data must be nearly current	Pilot light	Live replicated data; app servers started on failover
Bank internet banking; down only a few minutes	Warm standby	A small working copy only needs scaling up
Card payment network; no downtime allowed	Multi-site active/active	Both sites serve traffic all the time

9. Hardware infrastructure for high availability (slides 62–64)

- **High-availability storage**: storage HA is key to infrastructure HA. Use **redundant I/O paths** from servers to the storage array: server ↔ **HBAs** (host bus adapters: the server's storage network cards) ↔ Fibre Channel switches ↔ storage controllers ↔ disk enclosures. Two of everything, cross-connected, so one failed cable, card, switch or controller does not stop access.
- **Network HA**: fully redundant interconnects for clusters; **network bonding/teaming** (two NICs act as one link; if one fails the other carries traffic) vs Oracle **HAIP** (Highly Available Virtual IP); dedicated switches for private interconnects; link aggregation groups.

- **Oracle HA example (slide 64)**: clients → WAN traffic manager → application servers at a primary and a secondary site; RAC databases (Real Application Clusters: several servers running one database) at the primary; **Oracle Data Guard** ships changes over a dedicated network to physical and logical standby databases at the secondary site.

10. Exam-style questions with model answers

Q1. What is Network Functions Virtualization? State four advantages. (6 marks)
NFV decouples network functions such as firewalls, routers and encryption from dedicated proprietary hardware and runs them as software on virtual machines on standard commodity servers, reducing cost and accelerating service deployment; a new function is added by spinning up a VM. Advantages: reduced space for network hardware, reduced power consumption, reduced maintenance and hardware costs, easier upgrades, longer hardware life cycles, management from anywhere in the operator's network.

Q2. Distinguish NFV from SDN. (4)
NFV virtualizes individual network functions (they become software on VMs instead of appliances). SDN separates the control plane from the data plane and centralizes control in a software controller that programs the forwarding devices. They are complementary: SDN can steer traffic between virtualized functions.

Q3. Explain the "works on my machine" problem and how containers solve it. (4)
Software works on the developer's machine but fails elsewhere because of different library versions, configurations or OS settings (dependency conflicts, inconsistent environments). A container packages the application with all its dependencies into an image, so the same image runs identically in development, testing and production.

Q4. Describe the Docker architecture. Distinguish an image from a container. (6)
Docker uses a client-server model: the client sends commands (build, run, pull) to the Docker daemon, which builds images, runs containers, and manages networks, volumes and communication with registries. Registries (e.g. Docker Hub) store and share images. An image is a read-only template built from a Dockerfile in layers; a container is a running instance created from an image, and many containers can be created from one image.

Q5. Why is data stored in volumes rather than inside containers? (2)
Containers are ephemeral; when one is removed its internal file system is lost. Volumes store data outside the container so it persists across container removal and replacement.

Q6. Draw and explain a highly available web architecture on AWS. (8)
(Draw Figure 4.) A VPC spans two AZs. Each AZ has a public subnet with a NAT gateway and a private subnet with web front-end and app servers. An Application Load Balancer distributes requests across both AZs. The web and app tiers are in Auto Scaling groups spanning both AZs, which replace unhealthy instances and scale with demand. RDS runs a primary in AZ1 replicated to a secondary in AZ2. No tier has a single point of failure; losing an entire AZ still leaves a working copy.

Q7. Compare target tracking, simple and step scaling policies. (6)
Target tracking keeps a metric at a target value (e.g. CPU 50%), adding or removing instances automatically. Simple scaling performs one fixed action when a CloudWatch alarm fires (e.g. +1 instance at CPU > 70%). Step scaling uses banded ranges: the larger the breach, the larger the adjustment.

Q8. Define RTO and RPO. (4)
RTO (recovery time objective) is the maximum acceptable downtime before service is restored. RPO (recovery point objective) is the maximum acceptable data loss, measured as the time between the last recoverable copy and the disaster.

Q9. Compare the four disaster-recovery strategies. (8)
Backup & restore: only scheduled backups copied across regions, no standby; infrastructure built after the event; RPO/RTO hours; lowest cost. Pilot light: data replicated live, app tier idle; servers provisioned after the event; tens of minutes; low-medium cost. Warm standby: a scaled-down fully working copy always running; scaled up after the event; minutes; higher cost. Multi-site active/active: full production in both sites serving traffic; real-time recovery with near-zero data loss; highest cost.

Q10. Compare horizontal and vertical scaling with examples. (4)
Horizontal scaling adds or removes instances (scale out: 2 → 6 servers; scale in when demand drops); it improves

availability. Vertical scaling makes one instance more powerful (c3.large → c4.8xlarge, scale up); it has an upper limit and usually needs a restart.

Q11. Give two advantages of RDS over running a database on EC2, and one reason to choose EC2. (3)
RDS: AWS handles patching and maintenance; automated backups with point-in-time recovery and Multi-AZ failover. EC2: full shell access to the server and its dependencies, and lower usage cost if you accept the DBA work.

11. Common mistakes

Mistake	Correct idea
NFV = virtualizing the whole network	NFV virtualizes network **functions** only (the slide stresses this).
NFV and SDN are the same thing	NFV = functions as software; SDN = control plane separated and centralized.
Image and container used as synonyms	Image = read-only template; container = running instance of it.
Storing database files inside a container	Containers are ephemeral; use volumes for persistent data.
Swapping RTO and RPO	RTO = time to recover (downtime). RPO = point in time recovered to (data loss).
Pilot light = warm standby	Pilot light: only data live, servers off. Warm standby: small but fully working copy running.
"One AZ is enough if the server is big."	One AZ is a single point of failure; use at least two.
Vertical scaling = adding servers	Vertical = bigger instance; horizontal = more instances.
Putting databases in a public subnet	Keep app servers and databases in private subnets; only the load balancer faces the Internet.

Cheat sheet

NFV & SDN	
NFV	Decouple network functions (firewall, encryption, router) from dedicated HW; run on VMs on commodity servers. Functions only, not whole network
Pros	Less space, power, maintenance & HW cost; easier upgrades; longer HW life; manage from anywhere
Stack	MANO (orchestration) · VNFs · NFVI (+ SDN, VIM)
SDN	Control plane separated from data plane; central software controller
Why SDN	WAN cost, hybrid/multi-cloud control, prioritise critical traffic, new sites fast, security, multi-domain, future-proof

Docker	
Why	"Works on my machine", dependency conflicts → bundle app + deps; microservices; faster delivery
Isolation	Namespaces (own view) + cgroups (resource limits); shared host kernel
Parts	Client → daemon (build, run, networks, volumes) ↔ registry
Image	Read-only template; Dockerfile; 1 instruction = 1 cached layer
Container	Running instance; create, start, stop, restart, remove; ephemeral
Data	Volumes outside the container
Limit	Shared kernel ⇒ weaker isolation than VMs

AWS high availability	
Region	Geographic area, ≥ 2 AZs; pick near users
AZ	Cluster of DCs, own power/cooling, redundant network
HA design	VPC, 2 AZs, public subnet + NAT GW, private subnet web/app, ALB, Auto Scaling groups, RDS primary → secondary
3-tier	Web → app → database
Scaling	Out/in = horizontal; up = vertical
Launch template	Recommended over launch configs; versioned; advanced options
ASG	Right-size, logical group, spread over AZs, health checks; min/max/desired
Policies	Target tracking (keep target), Simple (CloudWatch alarm), Step (bands)
RDS	Managed, patching, PITR backups, snapshots, Multi-AZ

DR	Standby	RTO	$
Backup & restore	none	hours	$
Pilot light	data live, servers off	10s min	$$
Warm standby	small full copy	minutes	$$$
Multi-site	full, active/active	real time	$$$$
RPO = max data loss (time). **RTO** = max downtime.			